Féidearthachtaí as Cuimse
Infinite Possibilities

Machine Learning

Lecture 8: Reinforcement Learning

Bojan Božić & Bujar Raufi
School of Computer Science
TU Dublin, Grangegorman

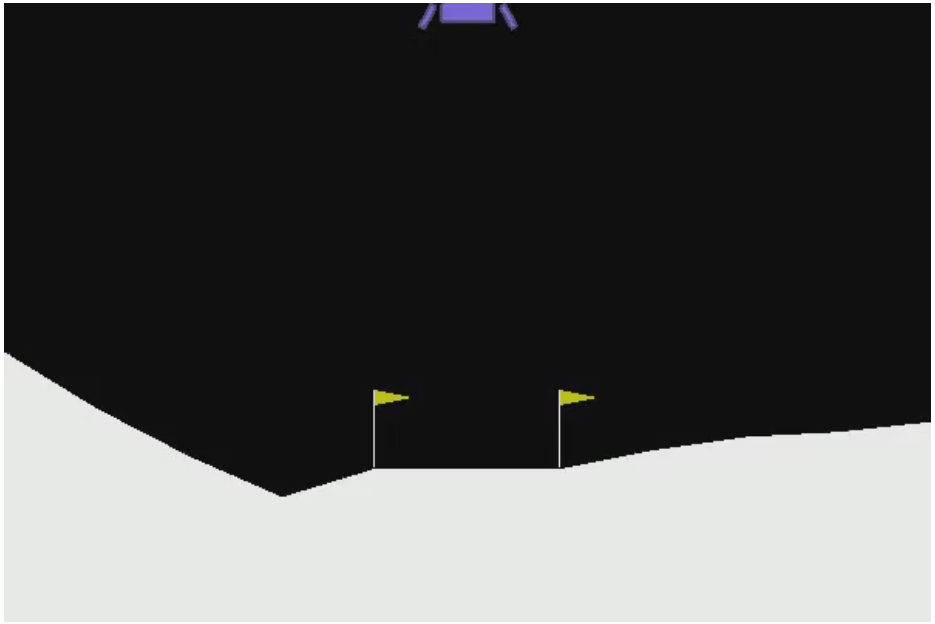
bojan.bozic@tudublin.ie; bujar.raufi@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH
TU DUBLIN
TECHNOLOGICAL UNIVERSITY DUBLIN

Slides adapted from https://huggingface.co/learn/deep-rl-course and Fundamentals of Machine Learning for Predictive Data Analytics. Kelleher, Mac Namee and D'Arcy

What is Reinforcement Learning?

RL is a type of Machine Learning where an agent learns **how to behave** in an environment **by performing actions** and **seeing the results.**

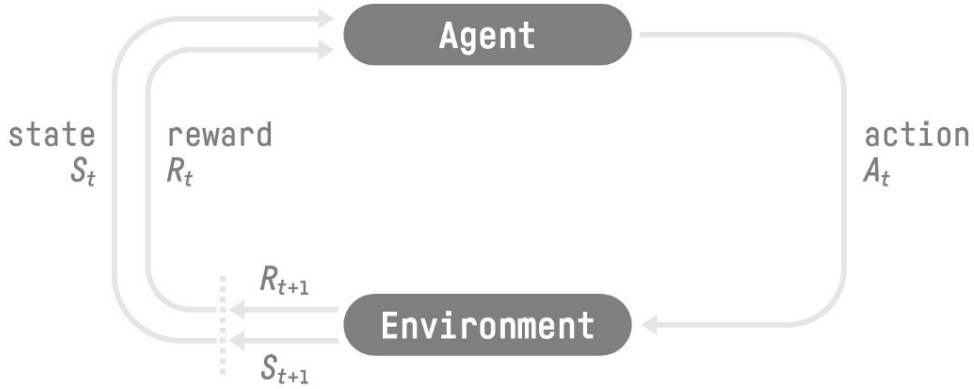


https://github.com/juliankappler/lunar-lander

Big Idea

- Sarah is a young venture scout in training for her pioneering badge.
- One of the more unusual challenges involved in earning this badge is to learn to cross a stream using a set of stepping-stones while wearing an electronic blindfold.
- The goal is to get across the river in the fewest steps possible without getting wet.
- Before the scout attempts a step, the blindfold is made transparent for 0.5 seconds to give the scout a quick view of their environment so that they make a decision about which direction they will step in and how far.

The RL Process



- Our Agent receives **state** S_0 from the **Environment** — we receive the first frame of our game (Environment).
- Based on that **state** S_0**,** the Agent takes **action** A_0 — our Agent will move to the right.
- The environment goes to a **new state** S_1 — new frame.
- The environment gives some **reward** R_1 to the Agent — we're not dead *(Positive Reward +1)*.

The agent's goal is to *maximize* its cumulative reward, **called the expected return.**

The Reward Hypothesis

the central idea of Reinforcement Learning

⇒ Why is the goal of the agent to maximize the expected return?

- Because RL is based on the **reward hypothesis**, which is that all goals can be described as the **maximization of the expected return** (expected cumulative reward).
- That's why in Reinforcement Learning, **to have the best behaviour,** we aim to learn to take actions that **maximize the expected cumulative reward.**

Observatsions/States Space

Observations/States are the **information our agent gets from the environment.** In the case of a video game, it can be a frame (a screenshot). In the case of the trading agent, it can be the value of a certain stock, etc.

- *State s*: is **a complete description of the state of the world** (there is no hidden information). In a fully observed environment.

In chess game, we receive a state from the environment since we have access to the whole check board information.

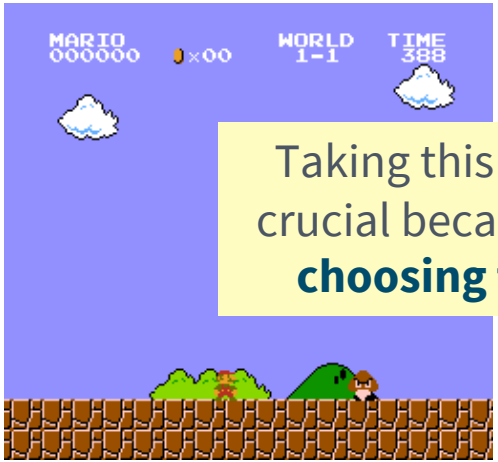
- *Observation o*: is a **partial description of the state.** In a partially observed environment.

In Super Mario Bros, we only see the part of the level close to the player, so we receive an observation.

Action Space

The Action space is the set of **all possible actions in an environment.**
The actions can come from a *discrete* or *continuous space*:

- *Discrete space*: the number of possible actions is **finite**.
- *Continuous space*: the number of possible actions is **infinite**.



In Super Mario Bros, we have only 4 possible actions: left, right, up (jumping) and down (crouching).

Taking this information into consideration is crucial because it will **have importance when choosing the RL algorithm in the future.**

A Self Driving Car agent has an infinite number of possible actions since it can turn left 20°, 21,1°, 21,2°, honk, turn right 20°…

Rewards and Discounting

The reward is fundamental in RL because it's **the only feedback** for the agent. Thanks to it, our agent knows **if the action taken was good or not.**

The cumulative reward at each time step **t** can be written as:

$$R(\tau) = r_{t+1} + r_{t+2} + r_{t+3} + r_{t+4} + \dots$$

Return: cumulative reward

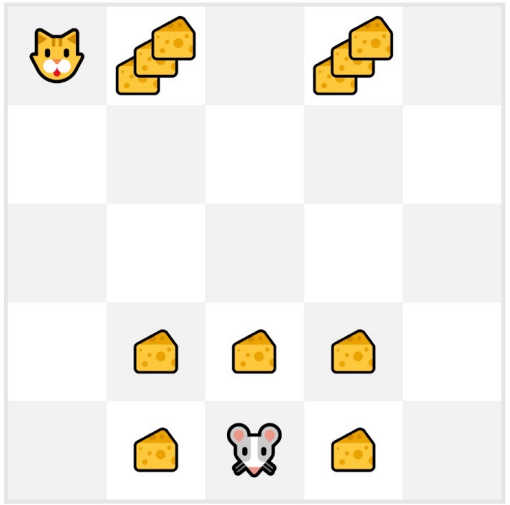
Trajectory (read Tau)
Sequence of states and actions

$$R(\tau) = \sum_{k=0}^{\infty} r_{t+k+1}$$

However, in reality, **we can't just add them like that.** The rewards that come sooner (at the beginning of the game) **are more likely to happen** since they are more predictable than the long-term future reward.

Rewards and Discounting

Let's say your agent is this tiny mouse that can move one tile each time step, and your opponent is the cat (that can move too). The mouse's goal is **to eat the maximum amount of cheese before being eaten by the cat.**



As we can see in the diagram, **it's more probable to eat the cheese near us than the cheese close to the cat** (the closer we are to the cat, the more dangerous it is).

Consequently, **the reward near the cat, even if it is bigger (more cheese), will be more discounted** since we're not really sure we'll be able to eat it.

Rewards and Discounting

To discount the rewards, we proceed like this:

1. We define a discount rate called gamma. **It must be between 0 and 1.** Most of the time between **0.95 and 0.99**.

- The larger the gamma, the smaller the discount. This means our agent **cares more about the long-term reward.**
- On the other hand, the smaller the gamma, the bigger the discount. This means our **agent cares more about the short term reward (the nearest cheese).**

2. Then, each reward will be discounted by gamma to the exponent of the time step. As the time step increases, the cat gets closer to us, **so the future reward is less and less likely to happen.**

Rewards and Discounting

Our discounted expected cumulative reward is:

$$R(\tau) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$

Return: cumulative reward

Gamma: discount rate

Trajectory (read Tau)
Sequence of states and actions

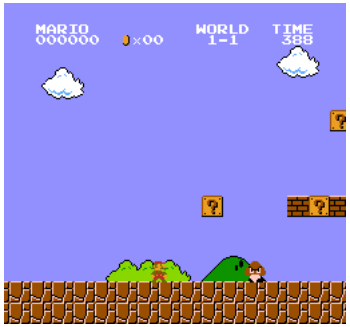
$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Type of Task

A task is an **instance** of a Reinforcement Learning problem. We can have two types of tasks:

Episodic task

Have a starting point and an ending point **(a terminal state). This creates an episode**: a list of States, Actions, Rewards, and new States.
For instance, think about Super Mario Bros: an episode begin at the launch of a new Mario Level and ends **when you're killed or you reached the end of the level.**



Continuing tasks

Continue forever (**no terminal state**). Agent must **learn how to choose the best actions and simultaneously interact with the environment.**
For instance, an agent that does automated stock trading. For this task, there is no starting point and terminal state. **The agent keeps running until we decide to stop it.**

The Exploitation/Exploration Tradeoff

Finally, before looking at the different methods to solve Reinforcement Learning problems, we must cover one more very important topic: *the exploration/exploitation trade-off.*

- *Exploration* is exploring the environment by trying random actions in order to **find more information about the environment.**
- *Exploitation* is **exploiting known information to maximize the reward.**

Remember, the goal of our RL agent is to maximize the expected cumulative reward. However, **we can fall into a common trap**.

Example

In this game, our mouse can have an **infinite amount of small cheese** (+1 each). But at the top of the maze, there is a gigantic sum of cheese (+1000).

However, if we only focus on exploitation, our agent will never reach the gigantic sum of cheese. Instead, it will only exploit **the nearest source of rewards,** even if this source is small (exploitation).

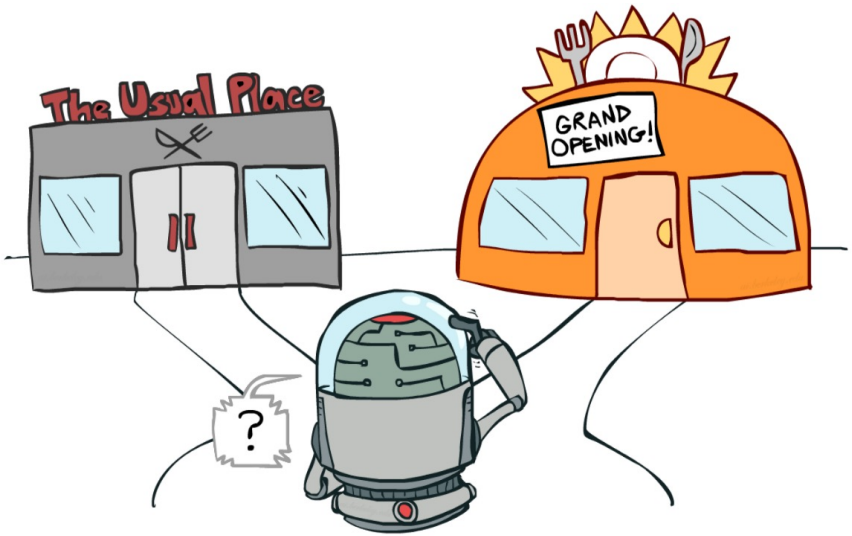
But if our agent does a little bit of exploration, it can **discover the big reward** (the pile of big cheese).

This is what we call the exploration/exploitation trade-off. We need to balance how much we **explore the environment** and how much we **exploit what we know about the environment.**

Example

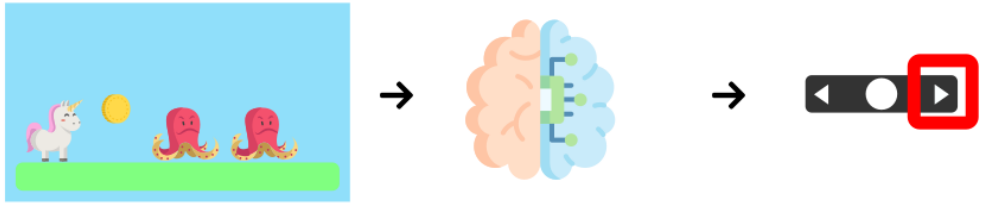
Therefore, we must **define a rule that helps to handle this trade-off**. We'll see the different ways to handle it in the future units.
If it's still confusing, **think of a real problem: the choice of picking a restaurant:**

- *Exploitation*: You go to the same one that you know is good every day and **take the risk to miss another better restaurant.**
- *Exploration*: Try restaurants you never went to before, with the risk of having a bad experience **but the probable opportunity of a fantastic experience.**



The Policy π: the agent's brain

The Policy **π** is the **brain of our Agent**, it's the function that tells us what **action to take given the state we are in.** It **defines the agent's behavior** at a given time.



State → π(State) → Action

Think of policy as the brain of our agent, the function that will tell us the action to take given a state

The Policy π: the agent's brain

This Policy **is the function we want to learn**, our goal is to find the optimal policy π*, the policy that **maximizes expected return** when the agent acts according to it. We find this π* **through training.**

There are two approaches to train our agent to find this optimal policy π*:

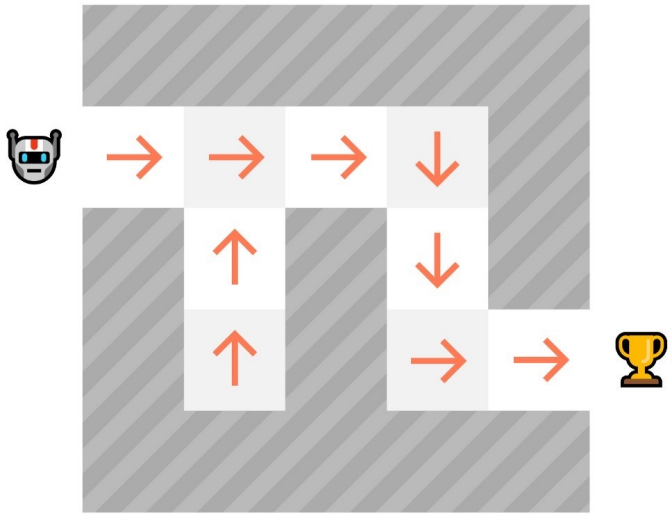
- **Directly,** by teaching the agent to learn which **action to take,** given the current state: **Policy-Based Methods.**
- Indirectly, **teach the agent to learn which state is more valuable** and then take the action that **leads to the more valuable states**: Value-Based Methods.

Policy-based Methods

In Policy-Based methods, **we learn a policy function directly.**

This function will define a mapping from each state to the best corresponding action. Alternatively, it could define **a probability distribution over the set of possible actions at that state.**

As we can see here, the policy (deterministic) **directly indicates the action to take for each step.**

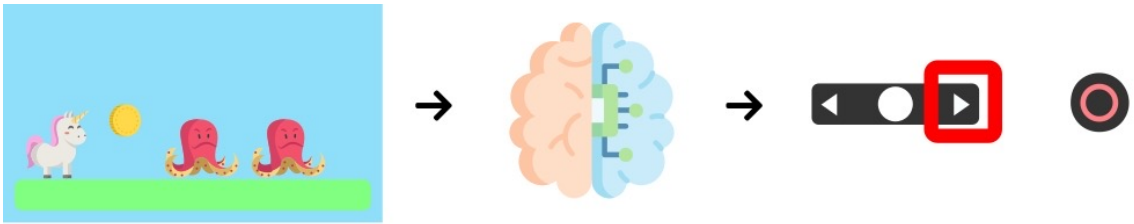


Deterministic Policy

a policy at a given state **will always return the same action.**

$$a = \pi(s)$$

action = policy(state)



State s_0 → $\pi(s_0)$ → a_0 = Right

Stochastic Policy

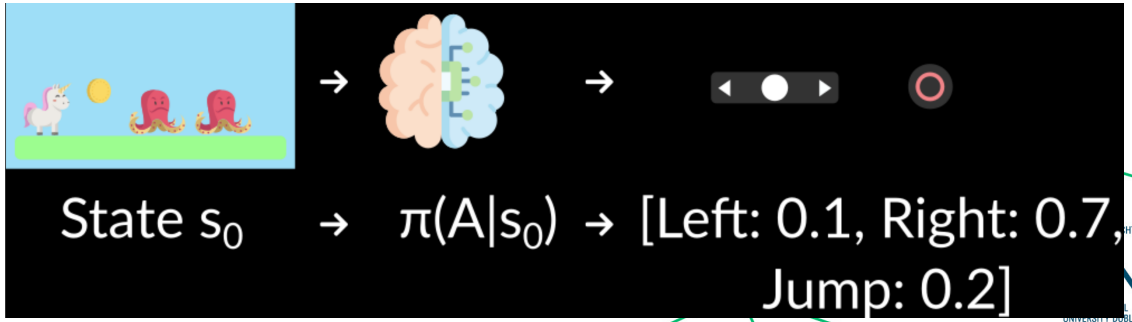
outputs **a probability distribution over actions.**

$$\pi(a|s) = P[A|s]$$

Probability Distribution over the set of actions given the state

policy(actions | state) = probability distribution over the set of actions given the current state

Given an initial state, our stochastic policy will output probability distributions over the possible actions at that state.



Value-based Methods

In value-based methods, instead of learning a policy function, we **learn a value function** that maps a state to the expected value **of being at that state.**

The value of a state is the **expected discounted return** the agent can get if it **starts in that state, and then acts according to our policy.**

“Act according to our policy” just means that our policy is **“going to the state with the highest value”.**

$$v_\pi(s) = \mathbb{E}_\pi\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \mid S_t = s\right]$$

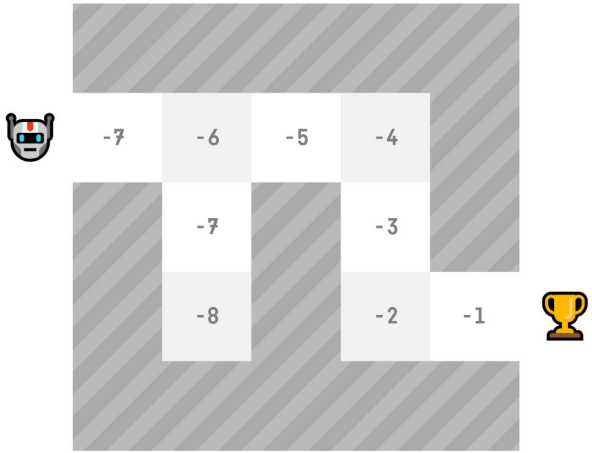
Value function

Expected discounted return

Starting at state s

Value-based Methods

Here we see that our value function **defined values for each possible state.**



Thanks to our value function, at each step our policy will select the state with the biggest value defined by the value function: -7, then -6, then -5 (and so on) to attain the goal.

The “Deep” in Reinforcement Learning

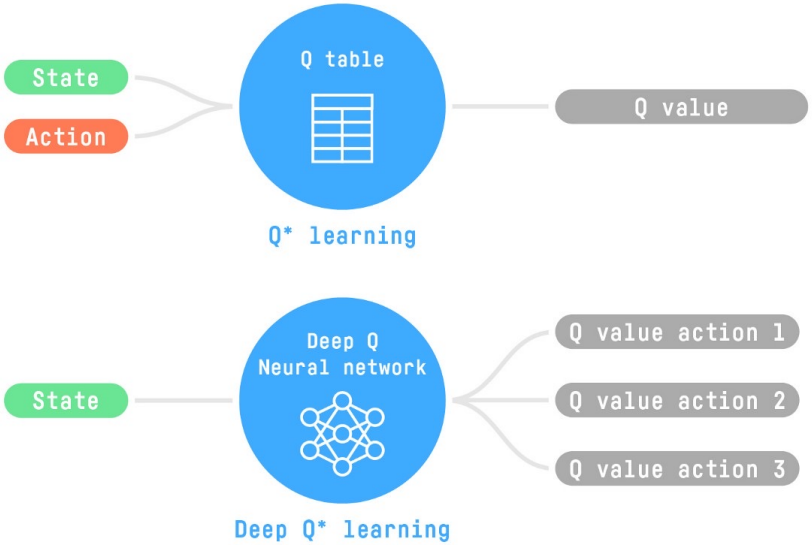
Deep Reinforcement Learning introduces **deep neural networks to solve Reinforcement Learning problems** — hence the name “deep”.

For instance, in the next unit, we’ll learn about two value-based algorithms: Q-Learning (classic Reinforcement Learning) and then Deep Q-Learning.

You’ll see the difference is that, in the first approach, **we use a traditional algorithm** to create a Q table that helps us find what action to take for each state.

The “Deep” in Reinforcement Learning

In the second approach, **we will use a Neural Network** (to approximate the Q value).



Summary

- Reinforcement Learning is a computational approach of learning from actions. We build an agent that learns from the environment **by interacting with it through trial and error** and receiving rewards (negative or positive) as feedback.
- The goal of any RL agent is to maximize its expected cumulative reward (also called expected return) because RL is based on the **reward hypothesis**, which is that **all goals can be described as the maximization of the expected cumulative reward.**
- The RL process is a loop that outputs a sequence of **state, action, reward and next state.**

Summary

- To calculate the expected cumulative reward (expected return), we discount the rewards: the rewards that come sooner (at the beginning of the game) **are more probable to happen since they are more predictable than the long term future reward.**
- To solve an RL problem, you want to **find an optimal policy**. The policy is the "brain" of your agent, which will tell us **what action to take given a state.** The optimal policy is the one which **gives you the actions that maximize the expected return.**
- There are two ways to find your optimal policy:
    1. By training your policy directly: **policy-based methods.**
    2. By training a value function that tells us the expected return the agent will get at each state and use this function to define our policy: **value-based methods.**

Questions?